

AI-Powered E-Commerce Retail Platform

Overview

Modern e-commerce platforms must serve millions of shoppers with personalised, low-latency experiences across product discovery, checkout, and post-purchase fulfilment. Retail teams also require operational visibility into inventory trends, anomalies, customer behaviour, and fulfilment bottlenecks. Build an AI-powered e-commerce retail platform that combines scalable commerce capabilities with Gen AI, NLP, and Agentic AI-driven customer and operational intelligence.

Sample queries your system should handle:

- I need a lightweight laptop for college under Rs.50,000 with good battery life — return ranked, relevant products.
- A customer asks why their order has not arrived three days after the expected delivery date.
- Surface trending products in electronics based on recent purchase and browsing patterns.
- Automatically flag inventory items likely to go out of stock within 48 hours given current sales velocity.

Requirement 1 (Basic)

- User management with account creation, login, and profile management; address book and saved payment methods.
- Product catalogue with search, filtering by category/brand/price, product detail pages with images, reviews, and ratings.
- NLP-enabled product search using semantic retrieval — map natural-language queries to structured filters.
- RAG-enabled shopping assistant for customer queries about products, availability, and recommendations.
- Shopping cart and checkout workflows with support for cards, wallets, and UPI.
- Order placement, tracking, cancellation, and refund workflows; notifications for order updates.
- Real-time inventory updates and low-stock alerts for operations teams.
- Recommendation engine for personalised product discovery.
- Tool-calling support for order lookup, inventory checks, and recommendations.
- Memory and session management for customer shopping interactions.
- Integration with payment and shipping providers.
- Input validation guardrails and basic anomaly detection.
- Simple front-end interface for customer interaction.

Requirement 2 (Advanced)

- Multi-agent orchestration for customer support, recommendations, fulfilment, and inventory planning.
- Hybrid search combining keywords, semantic retrieval, and behavioural signals with multi-turn query refinement.
- LLM-as-judge for validating product recommendation relevance and search result quality.
- DeepEval or equivalent evaluation framework for recommendation quality; custom metrics for cart conversion prediction and recommendation accuracy.

- Token optimisation for large-scale catalogue analysis and LLM-based attribute enrichment.
- Agent-to-agent communication between order management, inventory, and logistics agents to autonomously reroute orders when stock is unavailable.
- Root cause analysis for inventory mismatches and failed orders through collaborative AI agents.
- Demand forecasting using ML models to predict category-level demand and trigger automated restocking alerts.
- Feedback loop for recommendation refinement and customer behaviour adaptation.
- Anomaly detection for suspicious purchasing or inventory activities with real-time merchant-facing dashboards.

Non-Functional Requirements

NFR Dimension	Target / Requirement
Performance	Page load < 2 seconds; checkout flow < 5 seconds (P95)
Scalability	50,000 concurrent users; 500 million registered users
Availability	99.99% uptime; multi-region deployment; graceful degradation
Security	Secure payments (PCI-DSS); protect PII; prevent account takeover and fraud
Compliance	GDPR; PCI-DSS for payment data; consumer protection regulations
Reliability	Accurate inventory counts; idempotent order placement; no overselling
Observability	Monitoring, structured logging, anomaly detection; analytics dashboards
Maintainability	Modular microservices architecture; independent deployability per service
B2B / B2C	Consumer shopping flows and seller/merchant portal with catalogue management

Dataset

Dataset Name: Synthetic E-Commerce Retail Dataset

Format: CSV / JSON

Participants must create and use their own synthetic dataset simulating products, customer interactions, orders, inventory events, delivery updates, and recommendation data. A minimum of 5,000 products, 10,000 customers, and 50,000 orders is recommended.

Key Fields (minimum required schema):

Products

Field Name	Type	Description
product_id	UUID	Unique product identifier
category	string	Top-level category (e.g. Electronics, Clothing)
product_description	string	Long-form description used for NLP/semantic search
brand	string	Brand or manufacturer name

price	float	Listed price
discount_pct	float	Current discount percentage (0-100)
inventory_count	int	Available units in stock
ratings	float	Average customer rating (1.0-5.0)
customer_reviews	string	Aggregated review text or individual review records
tags	list[str]	Searchable attributes (e.g. wireless, lightweight)

Orders

Field Name	Type	Description
user_id	UUID	Reference to customer
order_id	UUID	Unique order identifier
order_status	enum	placed confirmed shipped delivered cancelled returned
cart_activity	JSON	Array of {product_id, quantity, unit_price}
payment_status	enum	paid pending failed refunded
shipment_status	enum	processing dispatched in_transit delivered failed
estimated_delivery	date	Expected delivery date
cancellation_reason	string	Free-text reason if cancelled or returned

You may extend these schemas with browsing events, wishlist, or search query logs to support recommendation and personalisation features. Document all extensions in your README.

Deliverables

1. Architecture Diagram

Include a high-level view of your system in JPEG or PDF. Show how data is processed from the synthetic dataset through ingestion, AI/ML layers, APIs, and the user-facing interface. Label all major components, data stores, and integration points.

2. Design

Articulate the system design decisions taken and the trade-offs made (e.g. choice of vector store, message broker, caching strategy, AI model selection, synchronous vs. asynchronous processing).

3. Full Executable Code (Microservice)

Focus on clarity and modularity. Provide a README explaining: (a) project setup — how to install dependencies and run the service; (b) sample usage — at least one end-to-end example showing a test input and the resulting output.

4. Panel Presentation (10 minutes)

Demo of your working solution outlining the design and approach (8 minutes). Q&A; (2 minutes).